

Glossário de Termos — Aula 03

Workflows Profissionais, Rebase e Pull Requests

Este glossário apresenta os termos avançados da Aula 03: rebase, cherry-pick, tags, Pull Requests, Git Flow e GitHub Actions. São conceitos usados no dia a dia de equipes profissionais de desenvolvimento.

Rebase

Rebase

Operação que reaplica os commits de uma branch sobre a ponta de outra, reescrevendo o histórico para criar uma linha reta. Resultado: histórico linear e mais limpo, sem commits de merge.

Analogia: Em vez de colar dois cadernos juntos, você recopia as páginas de um no final do outro — fica um caderno só, em ordem.

```
git switch feature/login  
git rebase main
```

Rebase Interativo (git rebase -i)

Modo avançado de rebase que permite editar, reordenar, combinar (squash) ou remover commits antes de reaplicá-los. Poderoso para limpar o histórico.

```
git rebase -i HEAD~3 → Edita os últimos 3 commits
```

Squash

Combinar múltiplos commits em um só durante um rebase interativo. Útil para transformar vários "work in progress" em um commit limpo.

Regra de Ouro do Rebase

NUNCA faça rebase em branches públicas/compartilhadas (como main). Rebase reescreve o histórico — se outros já puxaram esses commits, haverá conflitos graves. Use rebase apenas em branches locais/peçoais.

Analogia: Nunca reescreva uma página de um livro que já foi publicado — reescreva apenas seus rascunhos.

Merge vs Rebase

Merge preserva o histórico real (com commits de merge) — ideal para branches compartilhadas. Rebase cria histórico linear — ideal para branches pessoais antes de integrar ao main.

Cherry-pick

Cherry-pick

Comando que copia um commit específico de qualquer branch e aplica na branch atual, criando um novo commit com as mesmas mudanças mas hash diferente.

Analogia: Como pegar uma única receita de um livro de culinária e copiar no seu caderno — sem precisar trazer o livro inteiro.

```
git cherry-pick abc1234 → Aplica o commit abc1234 aqui
```

Tags e Versionamento

Tag

Marcador permanente associado a um commit específico. Usado para identificar versões de release do software. Diferente de branches, tags não se movem.

```
git tag v1.0.0 → Tag leve  
git tag -a v1.0.0 -m "Release 1.0.0" → Tag anotada
```

Tag Leve vs Tag Anotada

Tag leve é apenas um ponteiro para um commit (como um bookmark). Tag anotada é um objeto Git completo com autor, data e mensagem — recomendada para releases.

Semantic Versioning (SemVer)

Padrão de versionamento no formato MAJOR.MINOR.PATCH. MAJOR = mudanças incompatíveis, MINOR = novas funcionalidades compatíveis, PATCH = correções de bugs.

```
v1.0.0 → v1.1.0 (nova feature)  
v1.1.0 → v1.1.1 (bug fix)  
v1.1.1 → v2.0.0 (breaking change)
```

Pull Requests e Code Review

Pull Request (PR)

Solicitação no GitHub para integrar uma branch em outra. É o mecanismo principal de colaboração: permite discutir mudanças, revisar código e aprovar antes do merge.

Analogia: Como pedir para um colega revisar seu texto antes de publicar — ele pode sugerir mudanças ou aprovar.

Code Review (Revisão de Código)

Processo de revisão do código por outros desenvolvedores antes do merge. Identifica bugs, sugere melhorias e garante qualidade. Geralmente feito dentro de um Pull Request.

Aprovação (Approve)

Ação do revisor indicando que o código está pronto para merge. Muitas equipes exigem pelo menos uma aprovação antes de permitir o merge.

Request Changes

Ação do revisor solicitando modificações antes de aprovar. O autor do PR deve corrigir os pontos levantados e pedir nova revisão.

Git Flow e Workflows

Git Flow

Modelo de ramificação (branching model) que organiza o desenvolvimento com branches padronizadas: main (produção), develop (desenvolvimento), feature/*, release/*, hotfix/*.

Branch main (produção)

Contém apenas código estável e em produção. Cada merge em main geralmente corresponde a uma release com tag de versão.

Branch develop

Branch de integração onde as features são combinadas. Contém o código da próxima release em desenvolvimento.

Feature Branch

Branch criada a partir de develop para implementar uma funcionalidade específica. Após conclusão, é mergeada de volta em develop via Pull Request.

Release Branch

Branch criada a partir de develop quando o código está pronto para release. Permite ajustes finais sem bloquear novas features em develop.

Hotfix Branch

Branch de emergência criada a partir de main para corrigir bugs críticos em produção. Após a correção, é mergeada tanto em main quanto em develop.

GitHub Actions e CI/CD

GitHub Actions

Plataforma de automação integrada ao GitHub que executa workflows automaticamente em resposta a eventos (push, pull request, etc.). Usada para CI/CD, testes, deploy e automações diversas.

CI/CD (Integração Contínua / Entrega Contínua)

CI = rodar testes e validações automaticamente a cada push. CD = fazer deploy automaticamente quando o código é aprovado. Garante qualidade e agilidade na entrega.

Workflow (Fluxo de Trabalho)

Arquivo YAML em `.github/workflows/` que define quando e como executar tarefas automatizadas no GitHub Actions.

```
.github/workflows/ci.yml
```

Pipeline

Sequência automatizada de etapas (build, test, deploy) que o código percorre desde o commit até a entrega em produção.

Boas Práticas Profissionais

Commits Atômicos

Cada commit deve conter uma única mudança lógica e completa. Facilita revisão, bisect e reversão se necessário.

git stash

Salva temporariamente as mudanças não commitadas em uma pilha, limpando o working directory. Útil para trocar de branch rapidamente.

```
git stash → Guarda mudanças  
git stash pop → Restaura mudanças
```

git bisect

Ferramenta de busca binária para encontrar o commit que introduziu um bug. O Git testa commits automaticamente até isolar o problemático.

```
git bisect start  
git bisect bad → Commit atual tem bug  
git bisect good v1.0.0 → Essa versão funcionava
```

.gitkeep

Arquivo vazio usado por convenção para forçar o Git a rastrear uma pasta vazia (Git não rastreia pastas, apenas arquivos).